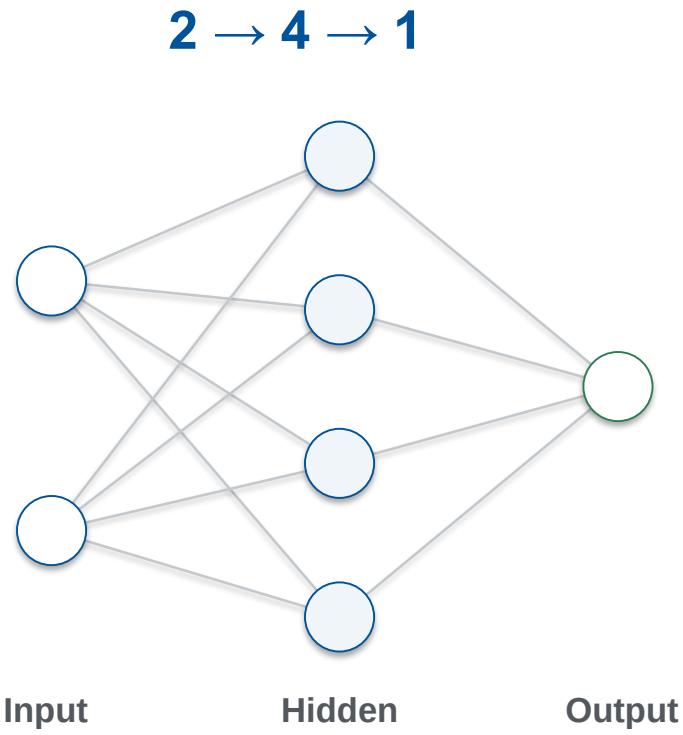
ANKARA UNIVERSITY

COM4573

ARTIFICIAL NEURAL NETWORKS

Colab Discussion • Week 3

Multilayer Perceptrons



Fall Semester 2026–2027

Instructor: Dr. Hilal Akarkamçı

Faculty of Engineering • Department of Computer Engineering

COLAB DISCUSSION

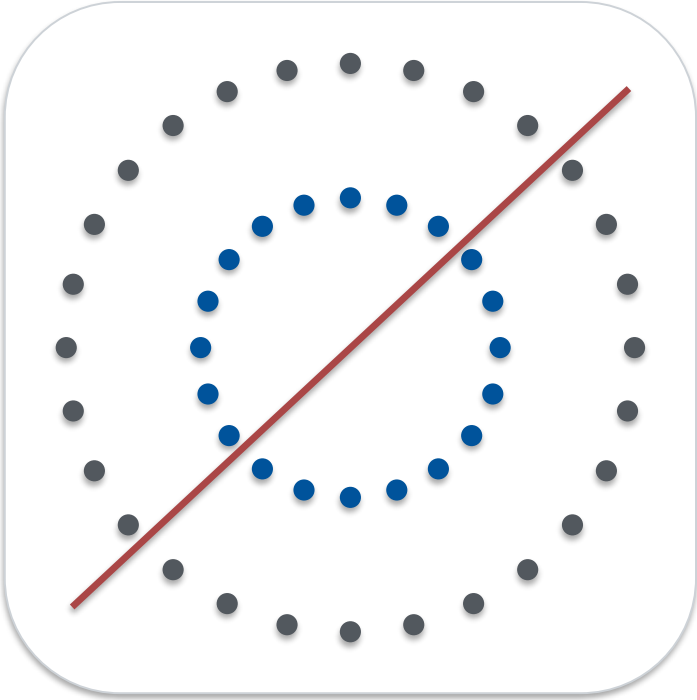
Why is this problem difficult for one perceptron?

QUESTION

Can one straight line separate the two classes?

EXPLANATION

In the concentric-circles dataset, one class lies near the center while the other surrounds it. A single perceptron creates one linear boundary, which divides the plane into two half-spaces. It cannot form a closed region around the inner class.



one linear boundary

Key idea **The limitation comes from the geometry of a linear classifier, not from the code.**

COLAB DISCUSSION

What does the Perceptron result tell us?

QUESTION

Why does changing w and b not fully solve this dataset?

EXPLANATION

The weights change the orientation of the line and the bias shifts its position. However, the decision boundary remains a line. Training can find a better linear separator, but it cannot change the model family into a nonlinear one.

$$z = w^T x + b$$

change w	change b
rotate	shift

Boundary is still linear

Key idea **Learning better parameters cannot remove a representational limitation.**

COLAB DISCUSSION

What changes when we add a hidden layer?

QUESTION

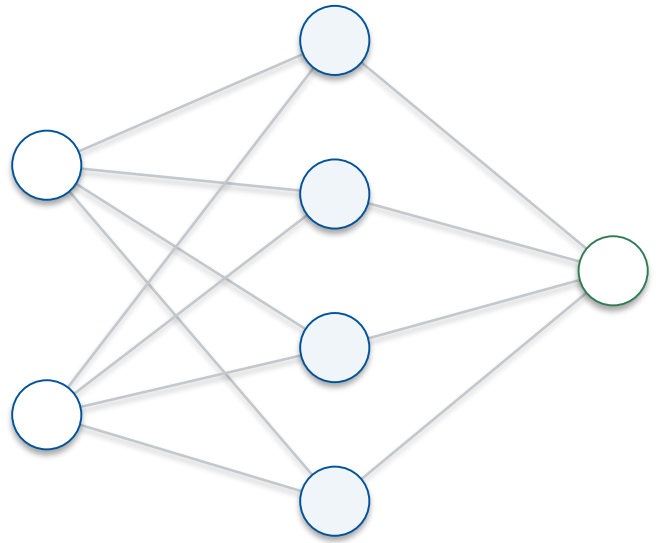
How can the MLP form a nonlinear decision region?

EXPLANATION

Each hidden neuron computes a weighted sum and applies ReLU. The hidden layer therefore creates several intermediate responses rather than one final linear test. The output neuron combines these responses to produce a more flexible decision function.

Key idea **Simple neurons can produce a complex boundary when their nonlinear outputs are composed.**

2 → 4 → 1



ReLU introduces the nonlinearity between layers.

COLAB DISCUSSION

Forward propagation • What happens inside the MLP?

QUESTION

What is the difference between z^1 and h^1?



EXPLANATION

z^1 is the hidden layer's pre-activation value: the weighted input plus bias. h^1 is the result after applying the activation function. With ReLU, negative z^1 values become zero while positive values pass forward.

$$z^1 = W^1x + b^1$$

$$h^1 = \text{ReLU}(z^1)$$

Key idea **Forward pass: $x \rightarrow z^1 \rightarrow h^1 \rightarrow z^2 \rightarrow \hat{y}$**

ReLU • Rectified Linear Unit

Definition

ReLU applies a simple rule to each neuron's pre-activation value z.

$$f(z) = \max(0, z)$$

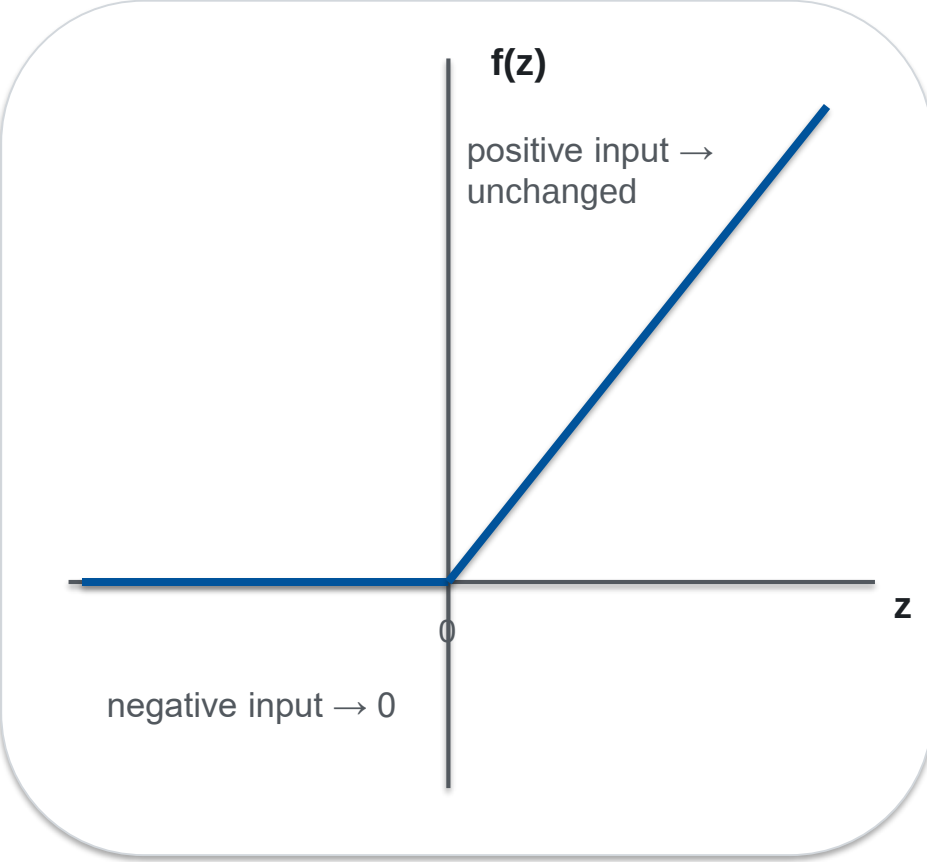
What does it do?

If $z \leq 0 \rightarrow$ output = 0
If $z > 0 \rightarrow$ output = z

Examples

ReLU(−2) = 0 **ReLU(0) = 0** **ReLU(3) = 3**

Key idea: ReLU introduces nonlinearity between Dense layers.



Why is ReLU important in an MLP?

Without a nonlinear activation, stacked Dense layers still collapse into one linear transformation.

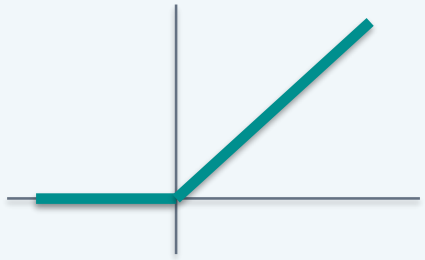
ReLU vs. Softmax — What is the difference?

COLAB DISCUSSION • IRIS MULTICLASS CLASSIFICATION

ReLU | Hidden layer

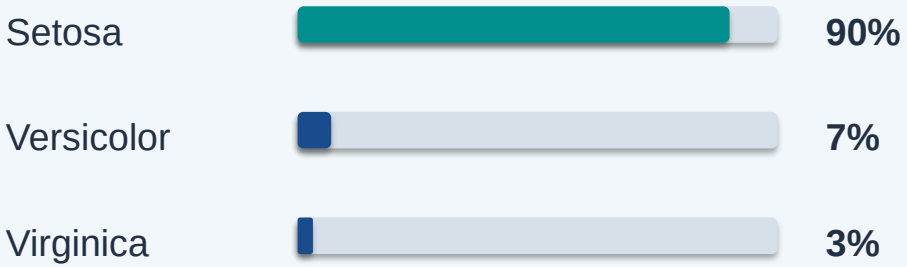
Introduces nonlinearity so the network can learn complex patterns and decision boundaries.

f(x) = max(0, x)



Softmax | Output layer

Converts class scores into probabilities that sum to 1.



Iris ANN: 4 input features → **Dense(8, ReLU)** → **Dense(3, Softmax)**

KEY TAKEAWAY **ReLU learns nonlinear representations; Softmax produces multiclass probabilities.**

COLAB DISCUSSION

Where do the 17 trainable parameters come from?

QUESTION

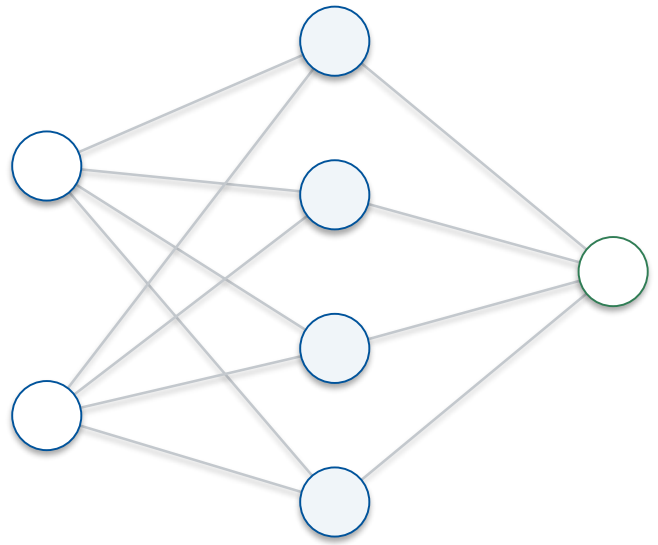
For the architecture 2 → 4 → 1, how many weights and biases are learned?

EXPLANATION

The hidden layer has 2×4 = 8 weights and 4 biases, giving 12 parameters. The output layer has 4×1 = 4 weights and 1 bias, giving 5 parameters. The complete network therefore has 17 trainable parameters.

Key idea **Dense-layer parameters = inputs × units + units**

2 → 4 → 1



Hidden: 8 + 4 = 12

Output: 4 + 1 = 5

Total = 17

COLAB DISCUSSION

Does a wider hidden layer always help?

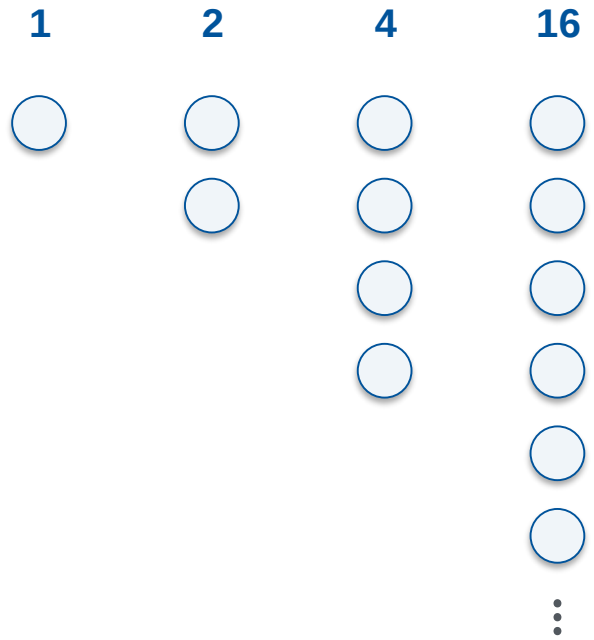
QUESTION

What changes when we compare 1, 2, 4, and 16 hidden neurons?

EXPLANATION

Increasing width gives the hidden layer more trainable units and can increase the flexibility of the decision function. It also increases the parameter count. More capacity, however, does not guarantee better performance on unseen data.

Key idea **Compare architectures using validation/test behavior, not size alone.**



More width → more parameters / capacity

When do we need a hidden layer?

A hidden layer becomes useful when the relationship between inputs and the target cannot be represented adequately by one linear transformation.

A linear model may be enough

- Classes are approximately linearly separable.
- The target changes roughly as a linear function of the inputs.
- A simple model already gives acceptable validation/test performance.

Example: one straight boundary can separate the classes.

A hidden layer may be needed

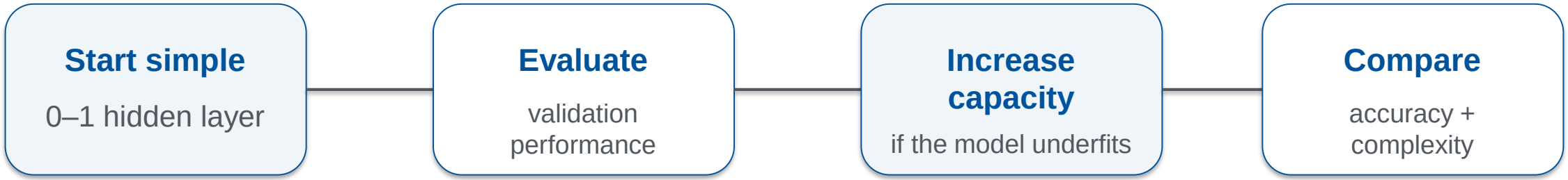
- One linear boundary cannot describe the pattern.
- Useful interactions between features must be learned.
- The task requires nonlinear or more complex intermediate representations.

Examples: XOR, concentric circles, complex sensor or image patterns.

Practical clue: if a well-trained linear baseline underfits, additional representational capacity may help.

Can we know the number of hidden layers in advance?

Usually, no. The dataset determines the input and output dimensions, but hidden depth and width are model-design choices.



What guides the choice?

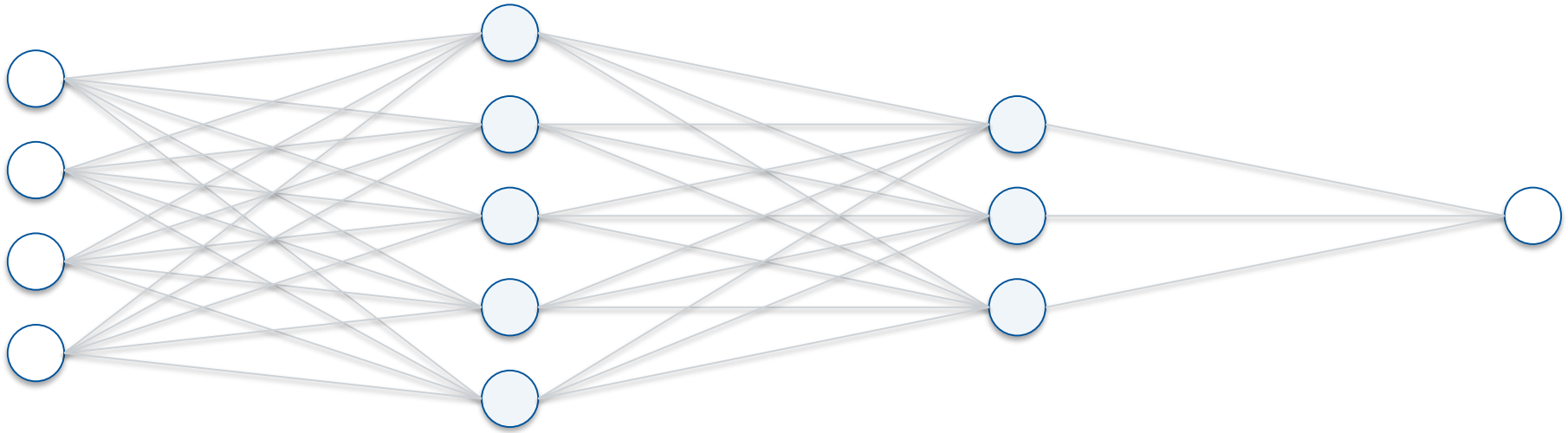
- validation performance
- underfitting / overfitting behavior
- amount of training data
- computational cost
- interpretability and deployment constraints

There is no universal rule such as "this problem needs exactly 3 hidden layers."

Architecture is normally treated as a hyperparameter and compared experimentally.

Hidden layer, Dense layer, depth and width

These terms describe different aspects of the network.



Input

Hidden 1
Dense(5)

Hidden 2
Dense(3)

Output
Dense(1)

Hidden layer — Any layer between input and output.

Dense layer — A fully connected layer type. A Dense layer can be hidden or output.

A practical starting strategy

For a small tabular classification problem, begin with the simplest reasonable network and add complexity only when the evidence supports it.

1	**Baseline**	Try a linear model or a very small MLP.
2	**One hidden layer**	Use a modest width; e.g., 4, 8, 16 or 32 units as candidates-not fixed rules.
3	**Validate**	Compare training and validation/test behavior.
4	**Adjust**	If underfitting persists, increase width or depth gradually.
5	**Stop**	Prefer the simpler architecture when added complexity gives no reliable benefit.

For our circles demo, 2 → 4 → 1 is a training choice that is small enough to inspect and flexible enough to demonstrate nonlinearity.